

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_0f586cf2-9eb\agent-acc49453d6495f8f5.jsonl`  
- SHA-256: `d7502bac918afe0b5b516fd43563ce05777c8f1751dd2f61d6332bd5b59b63b9`  
- Source modified: `2026-06-21T14:44:48+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_0f586cf2-9eb`  
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T14:42:18.602Z)

This is PR P0a from docs/PACKAGING_PLAN.md: introduce a resolved app-data HOME so a packaged app launched from an arbitrary CWD finds the same config/.env/DB/output, WITHOUT changing behaviour when RALLY_APP_HOME is unset. HARD CONTRACT: env unset => byte-identical to pre-P0a (CWD-relative). The repo: FastAPI engine, namespace package (no backend/__init__.py), hatchling editable install, windows-first users, requires-python>=3.10. Verified facts: full suite 1221 passed/7 skipped, ruff+mypy clean, coverage 85.10%, and an e2e proof showed config+DB land under RALLY_APP_HOME (nothing leaks to CWD) and == CWD when unset.

Changed/new files in the worktree C:/Users/avidu/Projects/bhi-wt-p0a:

- NEW backend/utils/app_home.py (the resolver:
app_home/app_path/default_config_path/default_env_path/resolve_output_dir/default_os_app_home)
- backend/config/loader.py (DEFAULT_CONFIG_PATH alias + use default_config_path())
- backend/main.py (load_dotenv prefers app-home .env; static mount package-relative; output_dir via resolve_output_dir at server-start + trim)
- scripts/doctor.py (writes config to default_config_path() not REPO_ROOT)
- NEW tests/test_app_home.py

LENS: cross-platform-paths

Read backend/utils/app_home.py. ADVERSARIALLY hunt cross-platform/Windows path bugs: expanduser on RALLY_APP_HOME; os.path.isabs semantics for a Windows drive-relative path like "C:foo" or a UNC path; str(Path(".")/"output")=="output" on Windows; default_os_app_home for Windows (%APPDATA%), macOS, Linux (XDG) ? are the leafs correct and is APP_DIR_NAME 'Khelsutra' fine? Any case where resolve_output_dir produces a surprising path? CRLF/encoding when save_default_config writes?

Read the actual files under C:/Users/avidu/Projects/bhi-wt-p0a. Return findings with severity blocker|major|minor|nit, each with file:line and a concrete fix. Be harsh but precise; do not invent issues.

2. assistant (2026-06-21T14:42:21.401Z)

I'll review the cross-platform path handling in the P0a worktree. Let me start by reading the key file.

3. user (2026-06-21T14:42:22.369Z)

```
1      """Resolved application-data HOME for a packaged, batteries-included install.  
2  
3      PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,  
4      the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and  
5      ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it  
6      from  
7      the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user  
8      silently
```

```

7 "loses" their index. This module introduces ONE resolver every state path roots on.
8
9 Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10 * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
sets it).
11 * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13 The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/local/share``)
14 is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15 calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16 this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17 real user profile during a test run).
18
19 stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20 the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21 """
22
23 from __future__ import annotations
24
25 import os
26 import platform
27 from pathlib import Path
28
29 #: Environment variable the packaged launcher sets to pin the app-data home.
30 APP_HOME_ENV = "RALLY_APP_HOME"
31
32 #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33 APP_DIR_NAME = "Khelsutra"
34
35
36 def app_home() -> Path:
37     """The resolved application-data home.
38
39     ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40 to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41     """
42     env = os.environ.get(APP_HOME_ENV)
43     if env and env.strip():
44         return Path(env.strip()).expanduser()
45     return Path(".")
46
47
48 def app_path(*parts: str | os.PathLike[str]) -> Path:
49     """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50     return app_home().joinpath(*[str(p) for p in parts])
51
52
53 def default_config_path() -> Path:
54     """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56     With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57     """
58     return app_home() / "config.json"
59
60

```

```

61     def default_env_path() -> Path:
62         """`<app-home>/env` ? the dotenv file the server prefers (e.g.
63         ``GEMINI_API_KEY``)."""
64         return app_home() / ".env"
65
66     def resolve_output_dir(output_dir: str) -> str:
67         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
68         through.
69         With ``RALLY_APP_HOME`` unset, ``resolve_output_dir("output") == "output"`` (CWD-
70         relative),
71         byte-identical to today; with it set, ``"output"`` becomes ``<app-home>/output`` so
72         the SQLite
73         DB and produced reels live alongside the rest of the app's state.
74         """
75         if not output_dir:
76             return str(app_home() / "output")
77         if os.path.isabs(output_dir):
78             return output_dir
79         return str(app_home() / output_dir)
80
81     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
82         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
83         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
84
85         * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
86         * macOS ? ``~/Library/Application Support/<app_name>``
87         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/local/share/<app_name>`` if
88         unset)
89         """
90         system = platform.system()
91         if system == "Windows":
92             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
93         elif system == "Darwin":
94             base = str(Path.home() / "Library" / "Application Support")
95         else:
96             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
97         return Path(base) / app_name

```

4. user (2026-06-21T14:42:22.731Z)

```

1     """Tests for the resolved application-data HOME (PACKAGING_PLAN.md P0a).
2
3     The load-bearing guarantee: with ``RALLY_APP_HOME`` UNSET, every resolver is byte-
4     identical to the
5     pre-P0a CWD-relative behaviour; with it SET, config/.env/output/DB all root under that
6     one home.
7     """
8
9     import os
10    from pathlib import Path
11
12    import pytest
13
14    from backend.utils import app_home as ah
15    from backend.config import loader
16
17    @pytest.fixture(autouse=True)
18    def _clear_app_home_env(monkeypatch):
19        """Each test starts with RALLY_APP_HOME unset unless it sets it explicitly."""
20        monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)

```

```

20
21
22 # ---- default (env unset) is byte-identical to today ----
23
24 def test_app_home_defaults_to_cwd():
25     assert ah.app_home() == Path(".")
26
27
28 def test_default_config_path_unset_is_cwd_config_json():
29     # Pre-P0a this was the module constant Path("config.json").
30     assert ah.default_config_path() == Path("config.json")
31
32
33 def test_default_env_path_unset_is_cwd_dotenv():
34     assert ah.default_env_path() == Path(".env")
35
36
37 def test_resolve_output_dir_unset_is_identity_for_relative():
38     assert ah.resolve_output_dir("output") == "output"
39     assert ah.resolve_output_dir("custom") == "custom"
40
41
42 # ---- RALLY_APP_HOME set ? everything roots there ----
43
44 def test_app_home_honors_env(monkeypatch, tmp_path):
45     monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
46     assert ah.app_home() == tmp_path
47
48
49 def test_app_home_expands_user(monkeypatch):
50     monkeypatch.setenv(ah.APP_HOME_ENV, "~/khelesutra-data")
51     assert ah.app_home() == Path.home() / "khelesutra-data"
52
53
54 def test_app_home_ignores_blank_env(monkeypatch):
55     monkeypatch.setenv(ah.APP_HOME_ENV, " ")
56     assert ah.app_home() == Path(".")
57
58
59 def test_config_and_env_paths_root_under_home(monkeypatch, tmp_path):
60     monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
61     assert ah.default_config_path() == tmp_path / "config.json"
62     assert ah.default_env_path() == tmp_path / ".env"
63     assert ah.app_path("output", "x.db") == tmp_path / "output" / "x.db"
64
65
66 def test_resolve_output_dir_set(monkeypatch, tmp_path):
67     monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
68     assert ah.resolve_output_dir("output") == str(tmp_path / "output")
69
70
71 def test_resolve_output_dir_absolute_passthrough(monkeypatch, tmp_path):
72     monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
73     abs_dir = str(tmp_path / "elsewhere") if os.path.isabs(str(tmp_path)) else
os.path.abspath("elsewhere")
74     assert ah.resolve_output_dir(abs_dir) == abs_dir
75
76
77 def test_resolve_output_dir_empty_falls_back_to_home_output(monkeypatch, tmp_path):
78     monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
79     assert ah.resolve_output_dir("") == str(tmp_path / "output")
80
81
82 # ---- default_os_app_home: the OS-conventional dir the launcher exports (NOT auto-
selected) ----

```

```

83
84     def test_default_os_app_home_windows(monkeypatch, tmp_path):
85         monkeypatch.setattr(ah.platform, "system", lambda: "Windows")
86         monkeypatch.setenv("APPDATA", str(tmp_path / "Roaming"))
87         assert ah.default_os_app_home() == tmp_path / "Roaming" / "Khelsutra"
88
89
90     def test_default_os_app_home_macos(monkeypatch):
91         monkeypatch.setattr(ah.platform, "system", lambda: "Darwin")
92         assert ah.default_os_app_home() == Path.home() / "Library" / "Application Support" /
"Khelsutra"
93
94
95     def test_default_os_app_home_linux_xdg(monkeypatch, tmp_path):
96         monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
97         monkeypatch.setenv("XDG_DATA_HOME", str(tmp_path / "share"))
98         assert ah.default_os_app_home() == tmp_path / "share" / "Khelsutra"
99
100
101     def test_default_os_app_home_linux_default(monkeypatch):
102         monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
103         monkeypatch.delenv("XDG_DATA_HOME", raising=False)
104         assert ah.default_os_app_home() == Path.home() / ".local" / "share" / "Khelsutra"
105
106
107     # ---- loader integration: load_config() roots on the app-home ----
108
109     def test_loader_reads_config_from_app_home(monkeypatch, tmp_path):
110         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
111         (tmp_path / "config.json").write_text('{"sport": "tennis"}', encoding="utf-8")
112         cfg = loader.load_config() # no explicit path ? default_config_path() under the
home
113         assert cfg.sport == "tennis"
114
115
116     def test_loader_no_config_in_home_uses_defaults(monkeypatch, tmp_path):
117         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path)) # empty dir, no config.json
118         cfg = loader.load_config()
119         # Defaults load cleanly (no crash, no file) ? same contract as a missing CWD
config.json.
120         assert cfg is not None
121         assert hasattr(cfg, "indexing")
122
123
124     def test_save_default_config_writes_under_app_home(monkeypatch, tmp_path):
125         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
126         written = loader.save_default_config() # no explicit path
127         assert written == tmp_path / "config.json"
128         assert written.exists()
129
130
131     def test_explicit_path_still_overrides_app_home(monkeypatch, tmp_path):
132         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path / "home"))
133         explicit = tmp_path / "explicit.json"
134         explicit.write_text('{"sport": "pickleball"}', encoding="utf-8")
135         cfg = loader.load_config(str(explicit))
136         assert cfg.sport == "pickleball"
137

```

5. assistant (2026-06-21T14:42:25.555Z)

Let me look at the consuming code to understand how `resolve\_output\_dir` and the path functions are actually used, since cross-platform bugs often manifest at the call sites.

6. user (2026-06-21T14:42:26.512Z)

```

1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from backend.utils.app_home import default_config_path
20
21     from .models import AppConfig
22     from .presets import (
23         COMPUTE_TARGET_FOR_PROFILE,
24         PROFILE_PRESETS,
25         is_known_profile,
26         preset_for,
27         suggest_profile,
28     )
29
30     logger = logging.getLogger(__name__)
31
32     # Back-compat alias (was a module constant; no external consumers). Prefer
33     # ``default_config_path()`` ? it honours ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a).
34     # With the # env unset this is ``Path("config.json")`` (CWD-relative), exactly the pre-P0a value.
35     DEFAULT_CONFIG_PATH = default_config_path()
36
37
38     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
39         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
40         scalars
41         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
42         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
43         profile
44         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
45         out = dict(base)
46         for key, val in override.items():
47             if isinstance(val, dict) and isinstance(out.get(key), dict):
48                 out[key] = _deep_merge(out[key], val)
49             else:
50                 out[key] = val
51         return out
52
53     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
54         """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
55         none
56         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
57         a bare
58         environment never silently selects a profile (and never silently spends on cloud).
59
60         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
61         file's
62         profile (rather than suppressing it) ? so an env typo can't leave the recorded
63         profile

```

```

59         asserting a preset that was never applied. A bad value in config.json is left to
surface as
60         a hard, typed-Literal error downstream (config-file correctness)."""
61         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
62         if env_profile and env_profile.strip():
63             ep = env_profile.strip()
64             if is_known_profile(ep):
65                 return ep
66             logger.warning(
67                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
68                 "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
69             )
70             # fall through to the file's profile (env typo must not suppress a valid file
choice)
71         dep = file_data.get("deployment")
72         if isinstance(dep, dict):
73             prof = dep.get("profile")
74             if isinstance(prof, str) and prof.strip():
75                 return prof.strip()
76         return ""
77
78
79     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"" ) -> list[str]:
80         """Dotted paths in `data` that the typed config will not honor.
81
82         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
83         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
84         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
85         user's intent is lost ? collect every such key so load_config can warn.
86         """
87         unknown: list[str] = []
88         fields = model_cls.model_fields
89         for key, val in data.items():
90             if key not in fields:
91                 unknown.append(prefix + key)
92                 continue
93             ann = fields[key].annotation
94             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
95                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
96         return unknown
97
98
99     def _get_builtin_defaults() -> dict[str, Any]:
100         """Return a plain dict of the current built-in defaults.
101
102         The AppConfig model is the single source of truth for defaults.
103         """
104         return AppConfig().model_dump(mode="json")
105
106
107     def load_config(path: str | Path | None = None) -> AppConfig:
108         """Load configuration with the following precedence:
109
110         1. Built-in defaults defined in the Pydantic models.
111         2. Values from the JSON file (if present and readable).
112         3. (Future) environment / CLI overrides.
113
114         Missing file or unreadable file ? returns a fully defaulted AppConfig.
115         Unknown keys in the file are tolerated (extra="allow" on the model).
116         """
117         cfg_path = Path(path) if path is not None else default_config_path()

```

```

118
119     file_data: dict[str, Any] = {}
120     if cfg_path.exists():
121         try:
122             with open(cfg_path, "r", encoding="utf-8") as f:
123                 file_data = json.load(f)
124         except Exception as exc:
125             # Non-fatal: continue with defaults + whatever we could parse.
126             # In production we might log here.
127             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
128
129     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
130     # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
131     # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
132     # loader's "bad input is non-fatal" contract.
133     if not isinstance(file_data, dict):
134         logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
135                        cfg_path, type(file_data).__name__)
136         file_data = {}
137
138     if file_data:
139         unknown = _unknown_key_paths(file_data, AppConfig)
140         if unknown:
141             logger.warning(
142                 "[CONFIG WARNING] %s contains keys the typed config does not "
143                 "recognize (typo'd or removed knobs ? top-level extras are kept "
144                 "but unread; unknown section keys are silently dropped): %s",
145                 cfg_path, ", ".join(sorted(unknown)),
146             )
147
148     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
149     # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
150     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
151     # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
152     defaults = _get_builtin_defaults()
153     profile = _resolve_explicit_profile(file_data)
154
155     if profile and not is_known_profile(profile):
156         # Only an unrecognised value from config.json reaches here (env typos already
fell back
157         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
158         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
159         logger.warning(
160             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
161             profile, ", ".join(PROFILE_PRESETS),
162         )
163         profile = ""
164
165     if profile:
166         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
167         # Record the profile actually applied ? the env may have chosen it over
config.json,
168         # so re-stamp it onto the merged deployment section to keep the record honest.

```

```

169         merged.setdefault("deployment", {})
170         if isinstance(merged["deployment"], dict):
171             merged["deployment"]["profile"] = profile
172             ct = merged["deployment"].get("compute_target")
173             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
174             if not ct and canonical:
175                 # An active profile with no explicit compute_target (e.g. config.json
blanked it
176                 # to "") gets its canonical target back ? the record never silently lies
about an
177                 # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
178                 merged["deployment"]["compute_target"] = canonical
179                 elif ct and canonical and ct != canonical:
180                     logger.warning(
181                         "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
182                         "config.json overrides it to %r ? honoring the explicit override.",
183                         profile, canonical, ct,
184                     )
185                     logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
186                                 profile, merged["deployment"].get("compute_target"))
187             else:
188                 # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
189                 merged = {**defaults, **file_data}
190                 suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
(D15)
191                 if suggestion:
192                     logger.debug(
193                         "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
194                         "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
195                         "auto-applies ? D15).", suggestion,
196                     )
197
198         return AppConfig.model_validate(merged)
199
200
201     def save_default_config(path: str | Path | None = None) -> Path:
202         """Write a fresh default config.json.
203
204         Used by scripts/doctor.py diagnostics and the --setup CLI path.
205         Overwrites if the file exists.
206         """
207         cfg_path = Path(path) if path is not None else default_config_path()
208         cfg = AppConfig()
209         cfg_path.parent.mkdir(parents=True, exist_ok=True)
210         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
211         return cfg_path
212
213
214     # Backwards-compatible alias used by some older call sites during transition.
215     def get_default_config() -> dict[str, Any]:
216         """Return defaults as a plain dict (for transitional use only)."""
217         return _get_builtin_defaults()
218

```

7. user (2026-06-21T14:42:28.607Z)

```

C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:13:from backend.utils.app_home import
default_env_path, resolve_output_dir
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:15:_env_path = default_env_path()
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:634:         out_dir =

```

```

resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-identical when unset)
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:670: # unset, resolve_output_dir("output")
== "output" (CWD-relative) ? byte-identical to before.
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:671: global_config.output.output_dir =
resolve_output_dir(args.output_dir)
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:19:from backend.utils.app_home
import default_config_path
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:33:# ``default_config_path()`` ? it
honours ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a). With the
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:35:DEFAULT_CONFIG_PATH =
default_config_path()
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:117: cfg_path = Path(path) if
path is not None else default_config_path()
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:207: cfg_path = Path(path) if
path is not None else default_config_path()
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:14:is computed by
:func:`default_os_app_home` but is **NOT** auto-selected here ? the launcher (P2)
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:32:#: OS-app-dir leaf used by
:func:`default_os_app_home` (the product brand).
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:36:def app_home() -> Path:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:48:def app_path(*parts: str |
os.PathLike[str]) -> Path:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:49: """A path under
:func:`app_home` (e.g. ``app_path("output")``)."""
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:50: return
app_home().joinpath(*[str(p) for p in parts])
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:53:def default_config_path() ->
Path:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:58: return app_home() /
"config.json"
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:61:def default_env_path() -> Path:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:63: return app_home() / ".env"
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:66:def
resolve_output_dir(output_dir: str) -> str:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:67: """Root a RELATIVE
``output_dir`` under :func:`app_home`; pass an ABSOLUTE one through.
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:69: With ``RALLY_APP_HOME``
unset, ``resolve_output_dir("output") == "output"`` (CWD-relative),
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:74: return str(app_home() /
"output")
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:77: return str(app_home() /
output_dir)
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:80:def
default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:82: ``RALLY_APP_HOME`` ? NOT
auto-selected by :func:`app_home`.
C:\Users\avidu\Projects\bhi-wt-p0a\scripts\doctor.py:52: from backend.utils.app_home
import default_config_path
C:\Users\avidu\Projects\bhi-wt-p0a\scripts\doctor.py:61: config_path =
str(default_config_path())
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:12:from backend.utils import app_home
as ah
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:17:def
_clear_app_home_env(monkeypatch):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:24:def
test_app_home_defaults_to_cwd():
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:25: assert ah.app_home() ==
Path(".")
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:28:def
test_default_config_path_unset_is_cwd_config_json():
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:30: assert ah.default_config_path()
== Path("config.json")
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:33:def
test_default_env_path_unset_is_cwd_dotenv():
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:34: assert ah.default_env_path() ==

```

```

Path(".env")
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:37:def
test_resolve_output_dir_unset_is_identity_for_relative():
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:38:    assert
ah.resolve_output_dir("output") == "output"
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:39:    assert
ah.resolve_output_dir("custom") == "custom"
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:44:def
test_app_home_honors_env(monkeypatch, tmp_path):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:46:    assert ah.app_home() ==
tmp_path
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:49:def
test_app_home_expands_user(monkeypatch):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:51:    assert ah.app_home() ==
Path.home() / "Khelsutra-data"
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:54:def
test_app_home_ignores_blank_env(monkeypatch):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:56:    assert ah.app_home() ==
Path(".")
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:61:    assert ah.default_config_path()
== tmp_path / "config.json"
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:62:    assert ah.default_env_path() ==
tmp_path / ".env"
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:63:    assert ah.app_path("output",
"x.db") == tmp_path / "output" / "x.db"
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:66:def
test_resolve_output_dir_set(monkeypatch, tmp_path):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:68:    assert
ah.resolve_output_dir("output") == str(tmp_path / "output")
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:71:def
test_resolve_output_dir_absolute_passthrough(monkeypatch, tmp_path):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:74:    assert
ah.resolve_output_dir(abs_dir) == abs_dir
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:77:def
test_resolve_output_dir_empty_falls_back_to_home_output(monkeypatch, tmp_path):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:79:    assert
ah.resolve_output_dir("") == str(tmp_path / "output")
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:82:# ---- default_os_app_home: the OS-
conventional dir the launcher exports (NOT auto-selected) ----
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:84:def
test_default_os_app_home_windows(monkeypatch, tmp_path):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:87:    assert ah.default_os_app_home()
== tmp_path / "Roaming" / "Khelsutra"
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:90:def
test_default_os_app_home_macos(monkeypatch):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:92:    assert ah.default_os_app_home()
== Path.home() / "Library" / "Application Support" / "Khelsutra"
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:95:def
test_default_os_app_home_linux_xdg(monkeypatch, tmp_path):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:98:    assert ah.default_os_app_home()
== tmp_path / "share" / "Khelsutra"
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:101:def
test_default_os_app_home_linux_default(monkeypatch):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:104:    assert
ah.default_os_app_home() == Path.home() / ".local" / "share" / "Khelsutra"
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:109:def
test_loader_reads_config_from_app_home(monkeypatch, tmp_path):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:112:    cfg = loader.load_config() #
no explicit path ? default_config_path() under the home
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:124:def
test_save_default_config_writes_under_app_home(monkeypatch, tmp_path):
C:\Users\avidu\Projects\bhi-wt-p0a\tests\test_app_home.py:131:def
test_explicit_path_still_overrides_app_home(monkeypatch, tmp_path):

```

Now let me examine the `main.py` usage around `default\_env\_path()` and the `output\_dir` resolution, plus how `output\_dir` flows into the DB path. The cross-platform concern about `resolve\_output\_dir` producing surprising paths depends on what consumes the result.

9. user (2026-06-21T14:42:33.347Z)

```
1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    # PACKAGING_PLAN.md P0a: prefer an app-home .env (the packaged launcher pins
RALLY_APP_HOME),
11    # else fall back to the historical CWD+walk-up search so the repo/dev path is unchanged.
12    # stdlib-only import ? keeps this pre-pydantic line light.
13    from backend.utils.app_home import default_env_path, resolve_output_dir
14
15    _env_path = default_env_path()
16    if _env_path.exists():
17        load_dotenv(_env_path)
18    else:
19        load_dotenv()
20
21    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
22    logging.basicConfig(
23        level=logging.INFO,
24        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
25        datefmt='%H:%M:%S'
26    )
27    logger = logging.getLogger(__name__)
28    from fastapi import FastAPI, HTTPException, Request
29    from fastapi.middleware.cors import CORSMiddleware
30    from pydantic import BaseModel
31    from typing import List, Dict, Any, Optional
32
33    from fastapi.staticfiles import StaticFiles
34    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
35    from backend.pipeline.validators.base import registry
36    from backend.infrastructure.database import Database
37    from backend.pipeline.segmenters.base import segmenter_registry
38    from backend.pipeline.stitcher.compiler import VideoCompiler
39    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
40    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
41    from backend.config import ( # new typed config
42        load_config as load_app_config,
43        AppConfig,
44        StitchingConfig,
45        enforce_startup_checks,
46        StartupCheckError,
47    )
48    from backend.results import Failure # standardized error/result contract
49    from backend.reporting import emit_indexer_report
50    from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
51    from backend import billing # M8: per-job cost ledger (USD internal / INR display)
52    from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
53
54    app = FastAPI(title="Sports Highlight Indexer API")
```

```

55
56
57     def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
58         """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
59         separated),
60         default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
61         ``backend.main``
62         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
63         its origin."""

```

10. user (2026-06-21T14:42:34.180Z)

```

620         return
621
622     if args.trim_start is not None and args.trim_end is not None:
623         if not args.video_path:
624             print("[ERROR] Please provide --video-path to extract a segment.")
625             return
626
627         # Determine output path
628         out_filename = args.trim_output or "trimmed_segment.mp4"
629         if os.path.isabs(out_filename):
630             out_path = out_filename
631             out_dir = os.path.dirname(out_path)
632             out_name = os.path.basename(out_path)
633         else:
634             out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
635             identical when unset)
636             out_path = os.path.join(out_dir, out_filename)
637             out_name = out_filename
638
639         os.makedirs(out_dir, exist_ok=True)
640         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
641         {args.trim_end}s from {args.video_path}...")
642
643         # global_config isn't initialized this early; load the typed config so the trim
644         # clip carries the same configured stitching paddings + watermark as a real
645         # highlight (watermark default-on).
646         stitching = load_app_config().stitching
647         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
648         end_padding=stitching.end_padding,
649         watermark=stitching.watermark)
650         try:
651             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
652             args.trim_end)], out_name)
653             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
654         except Exception as e:
655             print(f"[ERROR] Failed to compile trimmed segment: {e}")
656         return
657
658     if args.debug_clip is not None:
659         if not args.video_path:
660             print("[ERROR] Please provide --video-path to debug a specific clip.")
661             return
662         run_cli_diagnose_clip(args.video_path, args.debug_clip)
663         return
664
665     # Initialize Global Config and DB
666     global global_config, db_instance
667     global_config = load_app_config() # returns typed AppConfig
668     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
669     profile
670
671     # The CLI --output-dir overrides the configured output directory. It now lives
672     # on the typed model (OutputConfig.output_dir), so every consumer ? including

```

```

668         # /api/compile ? reads the same value instead of a write-only runtime hack.
669         # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
670         # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
671         global_config.output.output_dir = resolve_output_dir(args.output_dir)
672         os.makedirs(global_config.output.output_dir, exist_ok=True)
673
674         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
675         db_instance = Database(db_path)
676
677         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
678         # the executor is in-process. Mark them failed so pollers see a terminal state.
679         swept = db_instance.fail_stale_jobs()

```

11. assistant (2026-06-21T14:42:37.496Z)

Now I have full context on the call sites. Let me check what `args.output\_dir` defaults to, since `resolve\_output\_dir` receives it. This matters for the "empty string falls back to home/output" branch behavior.

12. user (2026-06-21T14:42:39.105Z)

```

13:from backend.utils.app_home import default_env_path, resolve_output_dir
283:         # report.output_dir). Never raises. Surface the paths in the response.
407:         (which only writes into output_dir; the browser previously got back an unreachable
415:         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
416:         path = os.path.join(output_dir, name)
418:         path = resolve_under_root(path, output_dir)
486:         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
487:         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
593:         parser.add_argument("--output-dir", default="output", help="Directory where processed
clips/highlights are saved")
604:         parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment
(saves in output-dir unless absolute path)")
634:         out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
666:         # The CLI --output-dir overrides the configured output directory. It now lives
667:         # on the typed model (OutputConfig.output_dir), so every consumer ? including
669:         # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
670:         # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
671:         global_config.output.output_dir = resolve_output_dir(args.output_dir)
672:         os.makedirs(global_config.output.output_dir, exist_ok=True)
674:         db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)

```

13. assistant (2026-06-21T14:42:45.255Z)

Now let me verify the key cross-platform behaviors empirically on this Windows machine, since the lens is specifically about Windows path semantics.

14. user (2026-06-21T14:42:47.543Z)

```

<string>:13: SyntaxWarning: invalid escape sequence '\s'
<string>:13: SyntaxWarning: invalid escape sequence '\s'
<string>:18: SyntaxWarning: invalid escape sequence '\s'
<string>:18: SyntaxWarning: invalid escape sequence '\s'
1. str(Path(".").joinpath("output")) = 'output'
   str(Path(".").joinpath("output").joinpath("x.db")) = 'output\\x.db'
2. isabs("C:foo") = False
   isabs("C:
oo") = False

```

```

isabs("/foo") = False
isabs("\\server\\share") = False
isabs("
oo") = False
3. Path("./")/"C:foo" = 'C:foo'
Path("./")/"\\server\\share" = '\\server\\share'

```

15. assistant (2026-06-21T14:42:49.998Z)

The escaping got mangled by the shell. Let me write a clean script to a file and run it to get reliable results.

16. assistant (2026-06-21T14:42:56.711Z)

```

import os
from pathlib import Path

print("=== isabs semantics on this platform ===")
for s in ["C:foo", r"C:\foo", "/foo", r"\\server\share", r"\foo", "foo"]:
    print(f" isabs({s!r}) = {os.path.isabs(s)}")

print("\n=== str(Path('.') / X) ===")
for x in ["output", "C:foo", r"\\server\share", r"C:\foo", "/abs/unix"]:
    print(f" str(Path('.') / {x!r}) = {str(Path('.') / x)!r}")

print("\n=== resolve_output_dir behaviour (env UNSET) ===")
from backend.utils.app_home import resolve_output_dir, app_home
os.environ.pop("RALLY_APP_HOME", None)
for x in ["output", "C:foo", r"\\server\share", r"\foo", "/abs/unix", ""]:
    print(f" resolve_output_dir({x!r}) = {resolve_output_dir(x)!r}")

print("\n=== resolve_output_dir behaviour (env SET to C:\\AppData\\Khelsutra) ===")
os.environ["RALLY_APP_HOME"] = r"C:\AppData\Khelsutra"
for x in ["output", "C:foo", r"\\server\share", r"\foo", "/abs/unix", ""]:
    print(f" resolve_output_dir({x!r}) = {resolve_output_dir(x)!r}")

print("\n=== drive-relative as RALLY_APP_HOME itself ===")
os.environ["RALLY_APP_HOME"] = "C:foo"
print(f" app_home() = {app_home()!r} -> str = {str(app_home()!r)}")
os.environ["RALLY_APP_HOME"] = r"\\server\share\khel"
print(f" app_home() = {app_home()!r} -> str = {str(app_home()!r)}")

```

17. user (2026-06-21T14:42:56.726Z)

File created successfully at: C:/Users/avidu/Projects/bhi-wt-p0a/scratch/probe_paths.py (file state is current in your context ? no need to Read it back)

18. user (2026-06-21T14:42:59.594Z)

```

Exit code 1
=== isabs semantics on this platform ===
isabs('C:foo') = False
isabs('C:\\foo') = True
isabs('/foo') = False
isabs('\\\\server\\share') = True
isabs('\\foo') = False
isabs('foo') = False

=== str(Path('.') / X) ===
str(Path('.') / 'output') = 'output'
str(Path('.') / 'C:foo') = 'C:foo'
str(Path('.') / '\\server\\share') = '\\server\\share\\'
str(Path('.') / 'C:\\foo') = 'C:\\foo'
str(Path('.') / '/abs/unix') = '\\abs\\unix'

```

```

=== resolve_output_dir behaviour (env UNSET) ===
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\bhi-wt-p0a\scratch\probe_paths.py", line 13, in <module>
    from backend.utils.app_home import resolve_output_dir, app_home
ModuleNotFoundError: No module named 'backend'

```

19. assistant (2026-06-21T14:43:02.327Z)

The first two sections ran. I need to fix the import path. Let me set PYTHONPATH.

20. user (2026-06-21T14:43:03.305Z)

```

=== isabs semantics on this platform ===
isabs('C:foo') = False
isabs('C:\\foo') = True
isabs('/foo') = False
isabs('\\\\server\\share') = True
isabs('\\foo') = False
isabs('foo') = False

=== str(Path('.') / X) ===
str(Path('.') / 'output') = 'output'
str(Path('.') / 'C:foo') = 'C:foo'
str(Path('.') / '\\\\server\\share') = '\\\\server\\share\\'
str(Path('.') / 'C:\\foo') = 'C:\\foo'
str(Path('.') / '/abs/unix') = '\\abs\\unix'

=== resolve_output_dir behaviour (env UNSET) ===
resolve_output_dir('output') = 'output'
resolve_output_dir('C:foo') = 'C:foo'
resolve_output_dir('\\\\server\\share') = '\\\\server\\share'
resolve_output_dir('\\foo') = '\\foo'
resolve_output_dir('/abs/unix') = '\\abs\\unix'
resolve_output_dir('') = 'output'

=== resolve_output_dir behaviour (env SET to C:\AppData\Khelsutra) ===
resolve_output_dir('output') = 'C:\\AppData\\Khelsutra\\output'
resolve_output_dir('C:foo') = 'C:\\AppData\\Khelsutra\\foo'
resolve_output_dir('\\\\server\\share') = '\\\\server\\share'
resolve_output_dir('\\foo') = 'C:\\foo'
resolve_output_dir('/abs/unix') = 'C:\\abs\\unix'
resolve_output_dir('') = 'C:\\AppData\\Khelsutra\\output'

=== drive-relative as RALLY_APP_HOME itself ===
app_home() = WindowsPath('C:foo') -> str = 'C:foo'
app_home() = WindowsPath('//server/share/khel') -> str = '\\\\server\\share\\khel'

```

21. assistant (2026-06-21T14:43:13.554Z)

This is very revealing. Several real Windows-specific behaviors here. Let me analyze the critical findings carefully.

Key observations from the empirical probe:

1. `resolve_output_dir("/abs/unix")` UNSET ? `'\\abs\\unix'` ? but the UNSET contract requires `**byte-identical to pre-P0a**`. Pre-P0a the value was passed through unchanged (it was just `args.output_dir`, used directly). Now even when env is unset, the value goes through `str(app_home() / output_dir)` = `str(Path(".") / "/abs/unix")` which on windows mangles `'/abs/unix'` ? `'\\abs\\unix'`. Wait ? but `'/abs/unix'` is not `'isabs'` on Windows (`'isabs('/foo')==False'`), so it falls to the join branch. Let me verify this is actually a behavior change vs pre-P0a. This is the most important contract concern.
2. `resolve_output_dir("C:foo")` SET ? `'C:\AppData\Khelsutra\foo'` ? a Windows drive-relative path `'C:foo'` (meaning "foo relative to CWD on drive C") gets its drive stripped and

rerooted. This is the classic `C:\foo` footgun, but it's an edge case for output dirs.

3. `resolve_output_dir(r"\foo")` SET ? `'C:\foo'` ? a rooted-but-driveless path `\\foo` is treated as relative (`isabs=False`) then joined, and `Path('C:\AppData\Khelsutra') / '\foo'` ? `'C:\foo'` (the rooted component resets to drive root). Surprising but again edge.

Let me focus on whether #1 is a genuine break of the hard contract. Let me check what pre-P0a did with `output_dir` ? was it ever passed through `str(Path...)`? Let me look at git history / the original main.py.

22. user (2026-06-21T14:43:15.232Z)

```
7835f34 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide (#291)
aac7633 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
(#290)
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching
(#289)
43eccc1 feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF)
(#288)
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)
(#287)
---DIFF main.py around output_dir vs master---
diff --git a/backend/main.py b/backend/main.py
index 0f48c7e..8e931f8 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -2,14 +2,21 @@ import os
import sys
import argparse
import logging
-import threading
-import traceback
import uuid
-from concurrent.futures import ThreadPoolExecutor
+from urllib.parse import quote
import uvicorn
from dotenv import load_dotenv

-load_dotenv()
+# PACKAGING_PLAN.md P0a: prefer an app-home .env (the packaged launcher pins RALLY_APP_HOME),
+# else fall back to the historical CWD+walk-up search so the repo/dev path is unchanged.
+# stdlib-only import ? keeps this pre-pydantic line light.
+from backend.utils.app_home import default_env_path, resolve_output_dir
+
+_env_path = default_env_path()
+if _env_path.exists():
+    load_dotenv(_env_path)
+else:
+    load_dotenv()

# Centralized logging setup for the indexer (hot paths use logger, CLI output uses print for
direct user-facing messages).
logging.basicConfig(
@@ -31,30 +38,55 @@ from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.utils.hashing import compute_video_id # canonical file-identity hashing
from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
-from backend.config import load_config as load_app_config, AppConfig, StitchingConfig # new
typed config
+from backend.config import ( # new typed config
+    load_config as load_app_config,
+    AppConfig,
+    StitchingConfig,
+    enforce_startup_checks,
+    StartupCheckError,
```

```

+)
    from backend.results import Failure # standardized error/result contract
    from backend.reporting import emit_indexer_report
+from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)
+from backend import billing # M8: per-job cost ledger (USD internal / INR display)
+from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-OFF)

    app = FastAPI(title="Sports Highlight Indexer API")

-# Enable CORS for the local web interface. allow_origins='' with allow_credentials=True is
-# rejected by browsers per the fetch spec and is an unsafe default to carry into the
-# auth/tenancy work; this tool uses no cookies, so credentials stay off.
+
+def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
+    """M9: CORS allow-origins from the `RALLY_CORS_ALLOW_ORIGINS` env var (comma-separated),
+    default `["*"]`. Read from the ENV (not a cwd `config.json`) so importing
+    `backend.main`
+    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
    origin."""
+    src = os.environ if env is None else env
+    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
+    origins = [o.strip() for o in raw.split(",") if o.strip()]
+    return origins or ["*"]
+
+
+## CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by browsers
per the
+## fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials
stay
+## off. The origin list is configurable via the env var above (default ["*"]) so a hosted deploy
can
+## lock it to its origin; the middleware is fixed at startup.
    app.add_middleware(
        CORSMiddleware,
        - allow_origins=["*"],
        + allow_origins=_cors_origins_from_env(),
        allow_credentials=False,
        allow_methods=["*"],
        allow_headers=["*"],
    )

-# Serves static frontend files directly (now under api/ per approved structure)
-os.makedirs("backend/api/static", exist_ok=True)
-app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
+## Serves static frontend files directly (now under api/ per approved structure).
+## PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from an
+## arbitrary directory finds the shipped assets instead of creating a stray
+## ./backend/api/static.
+## (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
+_STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
+os.makedirs(_STATIC_DIR, exist_ok=True)
+app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")

    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        - index_path = "backend/api/static/index.html"
        + index_path = os.path.join(_STATIC_DIR, "index.html")
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
@@ -64,18 +96,24 @@ def serve_index():
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None # now typed (Pydantic model)

-# Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single

```

```

-# self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
-# parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
-# already parallelize their AI handoff internally via their own pools.
-_job_executor: Optional[ThreadPoolExecutor] = None
-
-def _get_executor() -> ThreadPoolExecutor:
-    """Lazy module-global executor; tests monkeypatch this for inline execution."""
-    global _job_executor
-    if _job_executor is None:
-        _job_executor = ThreadPoolExecutor(max_workers=1, thread_name_prefix="jobworker")
-    return _job_executor
+
+## Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md P3)
+## now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
+## `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
+## `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams THROUGH
+## this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
+## intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
+## so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
+from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable on
+    backend.main)
+    JobWaiting,

```

23. assistant (2026-06-21T14:43:18.798Z)

Confirmed: pre-P0a, `args.output\_dir` (default `"output"`) was used directly without going through `str(Path(...))`. So the behavior change matters. Let me confirm what pre-P0a actually did ? was `output\_dir` assigned raw before? Let me look at the exact pre-P0a assignment.

24. user (2026-06-21T14:43:20.414Z)

```

227:         # report.output_dir). Never raises. Surface the paths in the response.
540:     (which only writes into output_dir; the browser previously got back an unreachable
548:     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
549:     path = os.path.join(output_dir, name)
551:     path = resolve_under_root(path, output_dir)
619:     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
620:     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
744:         out_dir = args.output_dir
776:     # on the typed model (OutputConfig.output_dir), so every consumer ? including
778:     global_config.output.output_dir = args.output_dir
779:     os.makedirs(global_config.output.output_dir, exist_ok=True)
781:     db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)

```

25. assistant (2026-06-21T14:43:26.864Z)

Confirmed. Pre-P0a (master line 778): `global\_config.output.output\_dir = args.output\_dir` ? the raw string, verbatim. And line 744: `out\_dir = args.output\_dir` raw.

Now the contract is "env unset => byte-identical to pre-P0a (CWD-relative)". With env unset, `resolve\_output\_dir(x)` returns:

```

- "output" ? "output" ? (identical)
- "custom" ? "custom" ?
- But "my/sub" ? str(Path(".") / "my/sub") = "my\sub" on Windows (separator flip), vs
pre-P0a verbatim "my/sub".
- And "/abs/unix" ? "\abs\unix".

```

Let me verify the forward-slash relative case precisely, which is the realistic contract-violation case (a user passing `--output-dir out/clips`).

26. user (2026-06-21T14:43:29.985Z)

```

pre-P0a raw='out/clips'           P0a-unset='out\clips'           identical=False

```

pre-P0a raw='a/b/c'	P0a-unset='a\\b\\c'	identical=False
pre-P0a raw='./output'	P0a-unset='output'	identical=False
pre-P0a raw='output/'	P0a-unset='output'	identical=False
pre-P0a raw='foo.bar'	P0a-unset='foo.bar'	identical=True

27. assistant (2026-06-21T14:43:39.821Z)

This confirms the **hard-contract violation**. With `RALLY_APP_HOME` unset` on Windows:

```
- `--output-dir out/clips` ? pre-P0a `"out/clips"`, P0a `"out\clips"` (separator flip ? NOT
byte-identical)
- `--output-dir ./output` ? pre-P0a `./output`, P0a `"output"` (normalization ? NOT byte-
identical)
- `--output-dir output/` ? pre-P0a `"output/"`, P0a `"output"` (trailing slash dropped)
```

The default `"output"` happens to round-trip, which is why the e2e proof and tests passed ? they only exercised the default. But any non-default relative dir with a separator or `./`` or trailing slash is silently rewritten. This breaks the stated **HARD CONTRACT** (`"env unset => byte-identical to pre-P0a"`).

Now let me check whether this is purely cosmetic or has functional consequences. The value flows into `db_path = os.path.join(output_dir, db_name)``, `os.makedirs(output_dir)``, and is stored in `global_config.output.output_dir`` which `/api/compile`` and `/api/clip`` read. On Windows `"out/clips"` and `"out\clips"` resolve to the same directory, so `makedirs/db` work the same. But the value is also returned to the browser/API and compared in some code paths. The real practical break is `./output`` and `output/`` normalization, and more importantly the contract is stated as `*byte-identical* ?` and it provably isn't.

Let me now double check the `default_os_app_home`` Windows leaf and the `app_path`` slash handling, and verify the macOS/Linux paths don't have separator issues (they're constructed via `Path`` joins so they're fine). Let me also confirm the `.env`` `load_dotenv`` change doesn't alter the unset behavior.

28. user (2026-06-21T14:43:40.899Z)

```
45         """Initialize a default config.json if it doesn't exist.
46
47         Now delegates to the canonical backend.config loader so that defaults
48         stay in sync with the Pydantic models (single source of truth).
49         """
50         try:
51             from backend.config import save_default_config
52             from backend.utils.app_home import default_config_path
53         except Exception as e:
54             print(f"[ERROR] Could not import backend.config loader: {e}")
55             print("[INFO] Falling back to no-op for config initialization.")
56             return
57
58         # PACKAGING_PLAN.md P0a: write to the SAME app-home config path the server READS
59         # (RALLY_APP_HOME, else CWD/config.json) ? previously this wrote to the repo root
while
60         # the server read from the CWD, so --setup and the server disagreed once CWD ? repo
root.
61         config_path = str(default_config_path())
62         if os.path.exists(config_path):
63             print(f"[INFO] {config_path} already exists. Skipping initialization.")
64             return
65
66         try:
67             saved_path = save_default_config(config_path)
68             print(f"[SUCCESS] config.json initialized with default settings at
{saved_path}.")
69         except Exception as e:
```

29. assistant (2026-06-21T14:43:50.769Z)

Now I have a complete picture. Let me consider the ``load_dotenv`` unset path: when ``RALLY_APP_HOME`` is unset, ``default_env_path()`` = ``Path(".env")``. If a ``.env`` exists in CWD, ``_env_path.exists()`` is True ? ``load_dotenv(Path(".env"))``. Pre-P0a was bare ``load_dotenv()``, which searches CWD ****and walks up parent directories****.

So there's a second subtle unset-behavior change: pre-P0a ``load_dotenv()`` walks up from CWD to find a ``.env`` in an ancestor dir; P0a, when a CWD ``.env`` does NOT exist, falls through to ``load_dotenv()`` (same walk-up) ? OK. But when a CWD ``.env`` DOES exist, both load it. The difference: if CWD ``.env`` exists, pre-P0a ``load_dotenv()`` also finds CWD ``.env`` first (find_dotenv starts at CWD). So that's actually equivalent. The walk-up only differs when CWD has no ``.env`` but an ancestor does ? and P0a correctly falls through to bare ``load_dotenv()`` in that case. So ``.env`` handling preserves the contract. Good ? not a finding.

Let me verify the CRLF/encoding question for ``save_default_config``: it uses ``write_text(..., encoding="utf-8")``. On Windows, ``Path.write_text`` opens in text mode, so ``\n`` in the JSON could translate to ``\r\n``. But ``model_dump_json`` produces ``\n`` separators; ``write_text`` text-mode on Windows applies newline translation. Let me check if that matters ? JSON readers tolerate CRLF, and the loader reads with ``encoding="utf-8"`` in text mode (universal newlines) so round-trips fine. Not a correctness bug, minor at most. Let me confirm whether ``write_text`` translates newlines here.

30. user (2026-06-21T14:43:53.035Z)

```
contains CRLF: True
repr: b'{"a": 1\r\n}'
```

31. assistant (2026-06-21T14:43:59.096Z)

Confirmed: on Windows ``write_text`` translates ``\n`` ? ``\r\n``. So ``save_default_config`` writes CRLF config.json on Windows. This is harmless for JSON parsing (the loader reads it fine, and JSON is newline-agnostic), but worth a nit since it's platform-divergent output. Not a real bug.

Now let me clean up the scratch file and finalize. Let me verify one more thing: the ``default_os_app_home`` Windows branch uses ``Path.home() / "AppData" / "Roaming"`` as fallback ? ``Path.home()`` on Windows returns ``C:\Users\<name>``, so that's correct. The leafs are all correct. ``APP_DIR_NAME = "Khelsutra"`` is a valid Windows/macOS/Linux directory name (no reserved chars). Fine.

Let me remove the scratch file.

32. user (2026-06-21T14:44:00.268Z)

removed

33. assistant (2026-06-21T14:44:08.216Z)

Let me do a final check on whether the contract claim in the module docstring is precisely worded, and reconsider severity. The docstring (line 9-11) says "DEFAULT IS BYTE-IDENTICAL TO TODAY" and ``resolve_output_dir`` docstring (line 69) explicitly claims ``resolve_output_dir("output") == "output"`` byte-identical. That specific example holds, but the general contract does not. Given the PR's stated HARD CONTRACT is "byte-identical to pre-P0a", and the realistic CLI input ``--output-dir out/clips`` provably diverges, this is a genuine contract gap. Whether it's a functional break: on Windows ``out/clips`` and ``out\clips`` reference the same directory, so files land correctly ? the divergence is in the **stored string value** echoed back via the API and config. So it's a real contract violation but low functional blast radius. I'll rate it major (contract is explicitly "byte-identical" and this falsifies it) rather than blocker.

Let me reconsider the absolute-path edge for ``/abs/unix`` and ``\foo``: these are unusual on Windows. The ``C:\foo`` drive-relative is genuinely a footgun but very rare as an output dir. I'll fold these into the ``resolve_output_dir`` finding.

Now finalizing my findings.

34. user (2026-06-21T14:44:41.965Z)

Structured output provided successfully

35. assistant (2026-06-21T14:44:48.403Z)

I've completed the adversarial cross-platform review of ``backend/utils/app_home.py`` and returned findings via `StructuredOutput`.

The headline finding (major): ``resolve_output_dir()`` routes the value through ``str(Path(".")) / output_dir)`` even when ``RALLY_APP_HOME`` is unset, which on Windows is not a verbatim identity ? ``out/clips`` becomes ``out\clips``, ``./output`` becomes ``output``, ``output/`` loses its trailing slash. The bare default ``"output"`` round-trips (which is why the tests and e2e proof passed), but the stated HARD CONTRACT of byte-identical-when-unset is provably false for any relative dir containing a separator/dot/trailing slash. I verified each case empirically against master's pre-P0a behavior (raw ``args.output_dir`` assignment). Two lower-severity items: Windows drive-relative ``C:foo``/driveless-rooted ``\foo`` get mis-rerooted when the env is set (because ``os.path.isabs`` is False for them), and ``save_default_config`` emits CRLF JSON on Windows (cosmetic). The OS-default leafs and ``APP_DIR_NAME`` are all correct.